EventFlow

Platforma distribuita de rezervare bilete la evenimente

Asistent laborator: Alexandru Tudor

Etapa 1 - Arhitectura proiectului

Deadline: 29 martie 2026

1. Informatii generale

Nume proiect:	EventFlow
Tema:	Platforma cloud-native pentru management evenimente, rezervari, plati si validare bilete

2. Formarea echipei si alegerea temei

Echipa de proiect

Membru 1:	Robert Barbu 343C1
Membru 2:	Vlad Grigore 343C1

Tema aleasa

Tema proiectului este dezvoltarea unei aplicatii de tip microservicii pentru:

- autentificare si autorizare utilizatori cu Keycloak (OIDC/OAuth2, RBAC)
- administrare si listare evenimente (creare, actualizare, disponibilitate)
- rezervare bilete, flux de plata cu sesiune temporara si lista de asteptare
- validare bilete la intrarea la eveniment (gate scanning)
- notificari asincrone dupa confirmarea platii

Descriere generala

EventFlow este o platforma distribuita pentru rezervarea biletelor la evenimente. Utilizatorul se autentifica prin auth-service, care integreaza Keycloak, navigheaza lista de evenimente disponibile, selecteaza un eveniment si cumpara un bilet printr-un flux de plata cu rezervare temporara de 2 minute. Dupa confirmarea platii, biletul devine activ si poate fi scanat la intrare de catre personalul evenimentului. Sistemul include lista de asteptare pentru evenimentele epuizate, rate limiting distribuit pentru protectia API-ului si caching Redis pentru performanta ridicata.

3. Arhitectura tehnica

3.1 Retele Docker Swarm (Overlay Networks)

Stack-ul foloseste patru retele overlay izolate:

- edge-network: Trafic dintre Kong API Gateway si microserviciile expuse public.
- service-network: Trafic intern intre microservicii, data-service, RabbitMQ si Redis.
- db-network: Acces la PostgreSQL permis exclusiv pentru data-service si pgAdmin.
- monitoring-network: Trafic intre Prometheus, Grafana si serviciile monitorizate.

Clusterul Docker Swarm este format din 3 noduri:

- 1 manager node responsabil pentru orchestrare si managementul clusterului
- 2 worker nodes pe care ruleaza microserviciile aplicatiei

Placement constraints:

- Serviciile aplicatiei (auth-service, data-service, user-profile-service, ticketing-service, payment-service, gate-service, notification-service) ruleaza pe worker nodes
- Componentele de infrastructura (PostgreSQL, pgAdmin, Portainer) ruleaza pe manager node

Replicile serviciilor sunt distribuite intre worker nodes pentru toleranta la defecte.

3.2 Microservicii proprii (Python / Flask)

auth-service (port 3003, 2 replici)

Autentificare utilizatori prin Keycloak (OIDC). Gestionare login/register, callback, sesiuni si token-uri JWT.

data-service (port 3002, 2 replici)

Serviciu dedicat accesului la date. Expune API CRUD pentru utilizatori, evenimente, bilete, plati si notificari. Singurul serviciu care comunica direct cu PostgreSQL.

user-profile-service (port 3004, 2 replici)

Profil utilizator si gestionare roluri (ADMIN/ORGANIZER/ATTENDEE/STAFF). Sincronizare automata cu Keycloak la primul acces. CRUD profil, adaugare/stergere roluri.

ticketing-service (port 3005, 2 replici)

API business pentru evenimente si bilete: creare/listare evenimente, cumparare bilet, lista de asteptare (waitlist), scanare, ban utilizatori. Rate limiting distribuit Redis (Lua scripting) + caching JSON cu TTL 5s.

payment-service (port 3008, 2 replici)

Initiere sesiune de plata cu rezervare temporara 2 minute, confirmare/anulare plata, creare bilet dupa confirmare. Rate limiting Redis. Publica ticket_booked in RabbitMQ.

gate-service (port 3007, 2 replici)

Validare bilete la intrarea la eveniment prin cod unic. Previne dubla scanare. Publica eveniment ticket_scanned in RabbitMQ.

notification-service (port 3006, 2 replici)

Consumer RabbitMQ (coada ticket_booked) rulat pe thread background. Persista notificari prin data-service si le expune prin API REST filtrat dupa rol.

3.3 Componente suport - infrastructura de date si mesagerie

- Keycloak 25.0: Identity Provider OIDC/OAuth2 cu RBAC, JWT, realm pre-configurat cu roluri si utilizatori de test.
- PostgreSQL 15: Persistenta relationala - baza de date aplicatie (eventflow) + baza de date dedicata

Keycloak.

- RabbitMQ 3.13: Broker mesaje asincrone: cozi ticket_booked si ticket_scanned.
- Redis 7: Rate limiting distribuit (ZSET + Lua scripting) si caching JSON (TTL 5s) pentru GET /events.

3.4 Componente cloud-native obligatorii

Conform cerintelor proiectului, stack-ul integreaza urmatoarele componente cloud-native:

- **Kong API Gateway**
  Expunere publica controlata a tuturor microserviciilor. Kong ruleaza in fata intregului cluster si aplica rutare unificata (rute declarate per serviciu), rate limiting la nivel de gateway, autentificare JWT plugin si logging centralizat. Toate requesturile externe trec exclusiv prin Kong.
- **Portainer**
  Gestiune vizuala a cluster-ului Docker Swarm: vizualizare servicii, stacks, containere, volume, retele si logs din browser. Accesat pe portul 9000 (UI) si 9443 (HTTPS).
- **pgAdmin 4**
  Administrare vizuala a bazelor de date PostgreSQL in mediu de dezvoltare si test. Permite inspectarea schemei, rularea query-urilor si exportul datelor.
- **Prometheus + Grafana**
  Sistem de monitorizare si observabilitate: Prometheus scrapeaza metricile expuse de microservicii (endpoint /metrics) la intervale de 15 secunde; Grafana ofera dashboards vizuale si alerte configurabile pe baza metricilor colectate.
- **GitHub Actions CI/CD**
  Pipeline automat declansat la fiecare push pe branch-ul main: build imagini Docker, rulare teste pytest, push imagini pe Docker Hub (idp/*), deploy automat in cluster prin docker stack deploy.

3.5 Flux de comunicare

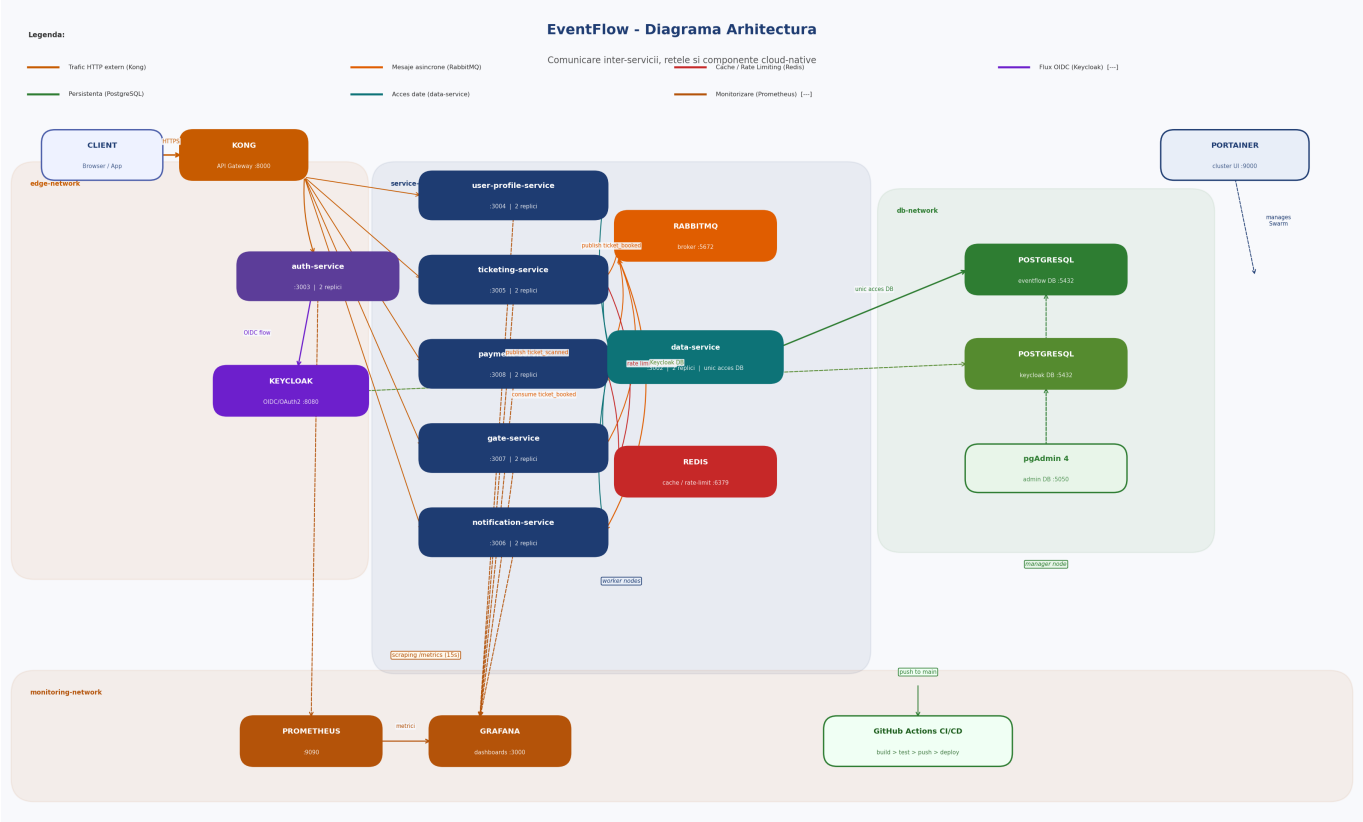
Serviciile comunica prin DNS intern Docker Swarm (rezolutie prin numele serviciului):

- Client extern -> Kong:8000 (singurul punct de intrare public, rutare catre microservicii)
- Kong -> auth-service (autentificare utilizatori)
- auth-service -> Keycloak:8080 (flux OIDC)
- user-profile-service -> data-service:3002
- ticketing-service -> data-service:3002
- payment-service -> data-service:3002
- gate-service -> data-service:3002
- notification-service -> data-service:3002
- data-service -> PostgreSQL:5432 (persistenta date)
- ticketing-service / payment-service -> Redis:6379
- ticketing-service / payment-service -> RabbitMQ:5672
- gate-service -> RabbitMQ:5672
- notification-service -> RabbitMQ:5672
- Prometheus -> toate microserviciile :*/metrics
- Grafana -> Prometheus:9090

3.6 Flux de plata (Payment Session - 2 minute)

1. POST /payments/start -> creeaza sesiune de plata cu expirare in 2 minute
2. Frontend afiseaza timer countdown de 2 minute
3. POST /payments/{id}/confirm -> confirma plata si creeaza biletul daca sesiunea e activa
4. POST /payments/{id}/cancel -> anuleaza sesiunea explicit
5. Dupa confirmare -> se publica eveniment ticket_booked in RabbitMQ
6. notification-service consuma evenimentul si persista notificarea prin data-service

Diagrama arhitectura - comunicare inter-servicii



4. Schema baza de date (PostgreSQL)

Tabel	Descriere
users	Profiluri utilizatori (keycloak_sub, email, name, timestamps)
user_roles	Roluri per utilizator (ADMIN, ORGANIZER, ATTENDEE, STAFF)
events	Evenimente (name, location, starts_at, total_tickets, tickets_sold, created_by)
tickets	Bilete cumparate (event_id, keycloak_sub, cod unic 32 char, used_at, used_by)
banned_users	Utilizatori blocati (keycloak_sub, reason)
waitlist_entries	Lista de asteptare (event_id, keycloak_sub, status, pozitie, promoted_ticket_id)
payment_sessions	Sesiuni plata (event_id, keycloak_sub, status, expires_at, ticket_id)
notifications	Notificari ticket_booked (event_id, organizer_sub, buyer_sub, cod, timestamp)

5. Limbaje si framework-uri pe servicii

Serviciu	Limbaj	Framework / Biblioteci principale
auth-service	Python 3.11	Flask, requests, PyJWT
data-service	Python 3.11	Flask, Flask-SQLAlchemy, psycopg2
user-profile-service	Python 3.11	Flask, Flask-CORS, PyJWT, requests
ticketing-service	Python 3.11	Flask, pika, redis, PyJWT, pytest, fakeredis, requests
payment-service	Python 3.11	Flask, pika, redis, PyJWT, pytest, fakeredis, requests
gate-service	Python 3.11	Flask, pika, PyJWT, requests
notification-service	Python 3.11	Flask, pika, PyJWT, requests

Tehnologii concrete folosite in platforma:

- Backend microservicii: Python 3.11 + Flask 3.0
- Persistenta relationala: PostgreSQL 15 (Alpine)
- Cache + Rate Limiting: Redis 7 (ZSET cu Lua scripting atomic)
- Mesagerie asincrona: RabbitMQ 3.13 (cozi durable, consumer pe thread background)
- Autentificare/Autorizare: Keycloak 25 (OIDC/OAuth2, JWT, RBAC, 4 roluri)
- API Gateway: Kong 3.7 (rutare, rate limiting, JWT plugin, logging)
- Gestiune cluster: Portainer CE (UI Docker Swarm)
- Administrare DB: pgAdmin 4
- Monitorizare: Prometheus + Grafana (scraping metrici, dashboards, alerte)
- CI/CD: GitHub Actions (build, test, push Docker Hub, deploy Swarm)
- Orchestrare containere: Docker + Docker Swarm (overlay networks, 2 replici per serviciu)
- Frontend: Vite + React (SPA)
- Testare: pytest + fakeredis (unit tests rate limiter si cache)

6. Roluri si permisiuni (RBAC Keycloak)

Rol	Permisiuni principale
ADMIN	Acces complet: gestionare utilizatori/roluri, bannare, toate notificarile/biletele
ORGANIZER	Creare/editare evenimente, gestionare waitlist, vizualizare notificari proprii

ATTENDEE	Cumparare bilete, vizualizare bilete proprii, intrare in lista de asteptare
STAFF	Scanare si validare bilete la intrarea la eveniment

Utilizatori de test pre-configurati in Keycloak

Username	Parola	Rol
admin1	password123	ADMIN
organizer1	password123	ORGANIZER
attendee1	password123	ATTENDEE
staff1	password123	STAFF

7. Impartirea task-urilor in echipa

Robert Barbu

- Arhitectura infrastructura Docker Swarm (docker-stack.yml, retele overlay, volume, deploy policies)
- Configurare Keycloak: realm, clienti OIDC, roluri, utilizatori de test, import automat la start
- Implementare user-profile-service (profil utilizator, roluri, sincronizare Keycloak)
- Implementare ticketing-service (evenimente, bilete, waitlist, ban, rate limiting Redis, caching)
- Implementare payment-service (sesiuni plata 2 min, confirmare, anulare, rate limiting Redis)
- Implementare gate-service (scanare bilete, validare cod unic, publicare RabbitMQ)
- Implementare notification-service (consumer RabbitMQ background, API notificari filtrat RBAC)
- Implementare rate_limiter.py si cache.py (Redis Lua scripting, unit tests pytest + fakeredis)
- Scrierea schemei SQL si a scripturilor de setup/build/test (setup.sh, build-images.sh, test-all.sh)

Vlad Grigore

- Configurare Kong API Gateway: rute, plugins JWT, rate limiting si logging la nivel de gateway
- Configurare Portainer pentru gestiunea vizuala a cluster-ului Docker Swarm
- Configurare pgAdmin 4 si conectare la bazele de date PostgreSQL din cluster
- Configurare Prometheus: scraping configuratie, targets pentru fiecare microserviciu
- Configurare Grafana: dashboards pentru latenta, throughput si starea serviciilor
- Implementare pipeline GitHub Actions CI/CD: build, test, push imagini Docker Hub (idp/*)
- Documentatie tehnica a proiectului si diagrama de arhitectura
- Testare integrare end-to-end a fluxului complet (autentificare, rezervare, plata, scanare)

8. DockerHub + GitHub - cont si repository-uri

Implementarea componentelor aplicatiei va fi realizata in repository-ul GitHub:

https://github.com/v-grigore/Proiect-IDP

Pentru etapa de containerizare si deployment, imaginile Docker aferente microserviciilor vor fi publicate in contul Docker Hub al echipei:

https://hub.docker.com/u/vladgrigore

Repository-uri imagini aplicative (custom)

Serviciu	Repository Docker Hub	Port
auth-service	https://hub.docker.com/r/vladgrigore/auth-service	3003
data-service	https://hub.docker.com/r/vladgrigore/data-service	3002
user-profile-service	https://hub.docker.com/r/vladgrigore/user-profile-service	3004
ticketing-service	https://hub.docker.com/r/vladgrigore/ticketing-service	3005
notification-service	https://hub.docker.com/r/vladgrigore/notification-service	3006
gate-service	https://hub.docker.com/r/vladgrigore/gate-service	3007
payment-service	https://hub.docker.com/r/vladgrigore/payment-service	3008

Componente suport (imagini publice Docker Hub / Quay.io)

Imagine	Rol
postgres:15-alpine	Baza de date relationala
quay.io/keycloak/keycloak:25.0.0	Identity Provider OIDC/OAuth2
rabbitmq:3.13-management	Message broker
redis:7-alpine	Cache si rate limiting
kong:3.7	API Gateway
portainer/portainer-ce:latest	Gestiune cluster Docker Swarm
dpage/pgadmin4:latest	Administrare PostgreSQL
prom/prometheus:latest	Colectare metrici
grafana/grafana:latest	Vizualizare metrici si dashboards

9. Concluzie

EventFlow este o platforma distribuita pentru rezervarea biletelor la evenimente, construita pe 7 microservicii Python/Flask care separa clar autentificarea, gestionarea evenimentelor, platile, validarea biletelor si notificarile. Autentificarea este gestionata printr-un serviciu dedicat (auth-service) care integreaza Keycloak, iar accesul la baza de date este centralizat prin data-service. Arhitectura respecta integral cerintele proiectului: microservicii autonome orchestrate cu Docker Swarm, expunere publica prin Kong API Gateway, gestiune vizuala prin Portainer, administrare DB prin pgAdmin, monitorizare prin Prometheus+Grafana si pipeline CI/CD automat prin GitHub Actions. RabbitMQ asigura comunicarea asincrona, Redis ofera rate limiting distribuit si caching, iar Keycloak gestioneaza OIDC/RBAC cu 4 roluri distincte. Fiecare serviciu custom ruleaza cu 2 replici, demonstrand scalabilitatea orizontala si rezilienta in caz de esec al unui container.